

01. Microtasks (Microtask Queue)

Ek line me

Promise ke `.then / .catch / .finally` handlers **hamesha baad me** chalte hain, current code khatam hone ke baad.

Samjho

js

```
let promise = Promise.resolve();
promise.then(() => console.log("promise done"));
console.log("code finished");
// Output: code finished -> promise done
```

Promise pehle se ready hai, phir bhi `.then` baad me chala. Kyun?

Microtask Queue kya hai?

- JS engine ke andar ek internal queue hoti hai (`PromiseJobs`), jise **microtask queue** kehte hain.
- Jab promise ready hota hai, uske `.then` handlers is queue me **daal diye jaate hain**, turant run nahi hote.
- Queue **FIFO** hai (jo pehle aaya, wo pehle chala).
- Engine jab current code se free hota hai, tab queue se ek-ek task uthata hai.

Order chahiye to?

`.then` chain karo:

js

```
Promise.resolve()
  .then(() => console.log("promise done"))
  .then(() => console.log("code finished"));
```

Unhandled Rejection

- Agar promise reject hua aur microtask queue khaali hone tak koi `.catch` nahi laga, to engine `unhandledrejection` event fire karta hai.
- Baad me (jaise `setTimeout` se) `.catch` lagane se bhi wo event nahi rukta, kyunki wo pehle hi fire ho chuka hota hai.

Yaad rakho

- Promise handlers hamesha async hote hain.
- Kisi code ko `.then` ke **baad** chalana ho to usse chained `.then` me daalo.
- Microtasks ka event loop / macrotasks se gehra connection hai (alag chapter me).